

Chapter 3: Processes

Operating System Concepts, 7th Edition — Silberschatz, Galvin, Gagne

1. Process Concept

An operating system executes a variety of programs. In batch systems these are called jobs; in time-shared systems they are called user programs or tasks. The textbook uses the terms job and process almost interchangeably.

A process is a program in execution. Process execution must progress in a strictly sequential fashion — at any instant, at most one instruction is executed on behalf of the process.

A process includes:

- Program counter — indicates the next instruction to execute
- Stack — holds temporary data (function parameters, return addresses, local variables)
- Data section — holds global variables

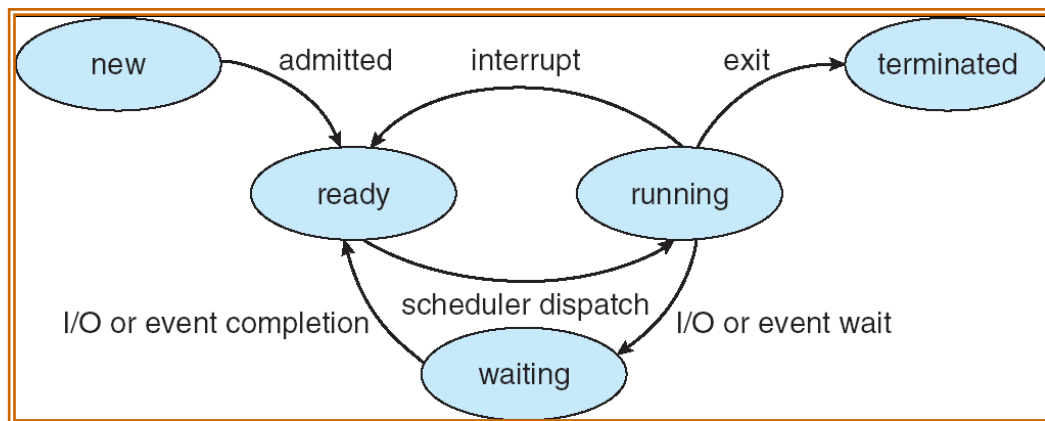
A process's memory layout (low to high address) is: text (program code), data (global variables), heap (dynamically allocated memory, grows upward), and stack (grows downward toward the heap).

2. Process State

As a process executes, it changes state. The five states are:

- New — the process is being created
- Running — instructions are being executed
- Waiting — the process is waiting for some event to occur (e.g., I/O completion)
- Ready — the process is waiting to be assigned to a processor
- Terminated — the process has finished execution

Transitions: New → (admitted) → Ready → (scheduler dispatch) → Running → (exit) → Terminated. From Running, a process can go to Waiting (I/O or event wait) and back to Ready (I/O or event completion), or back to Ready directly on an interrupt.



3. Process Control Block (PCB)

The PCB is a data structure that stores all the information associated with a specific process, enabling the OS to manage and switch between processes. It contains:

- Process state
- Process number (PID)
- Program counter

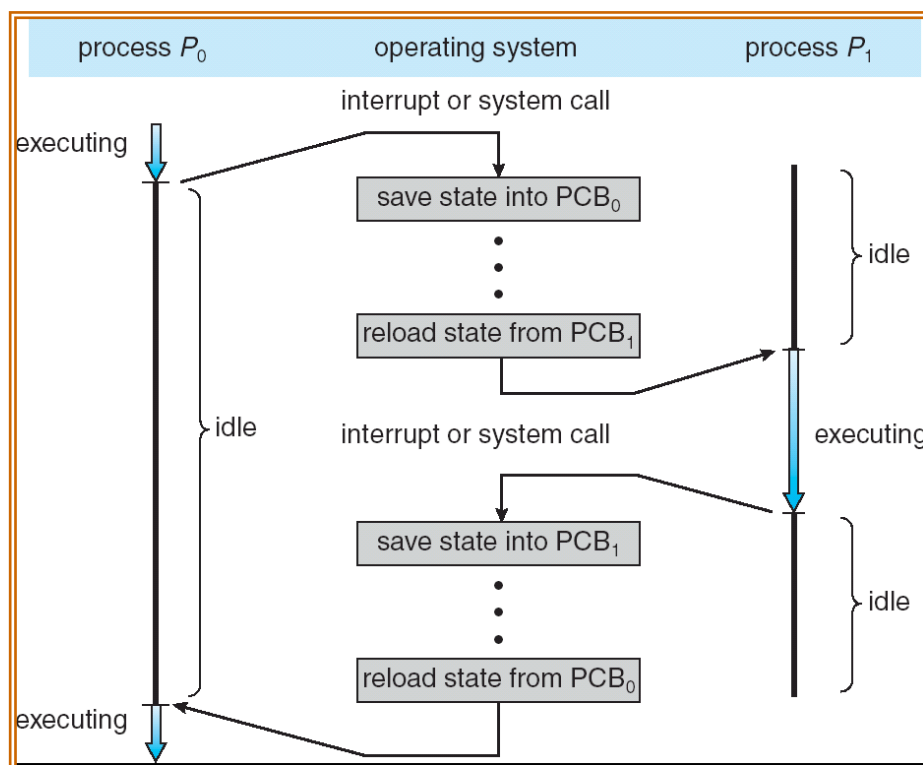
- CPU registers
- CPU scheduling information
- Memory-management information (memory limits, page tables, etc.)
- Accounting information
- I/O status information (list of open files, allocated I/O devices)



4. CPU Switch Between Processes / Context Switch

When the CPU switches to another process, the system must save the state of the old process (into its PCB) and load the saved state of the new process (from its PCB). This is called a context switch.

- Context-switch time is pure overhead — the system does no useful work while switching
- Switch time is dependent on hardware support (number of registers, special instructions, etc.)

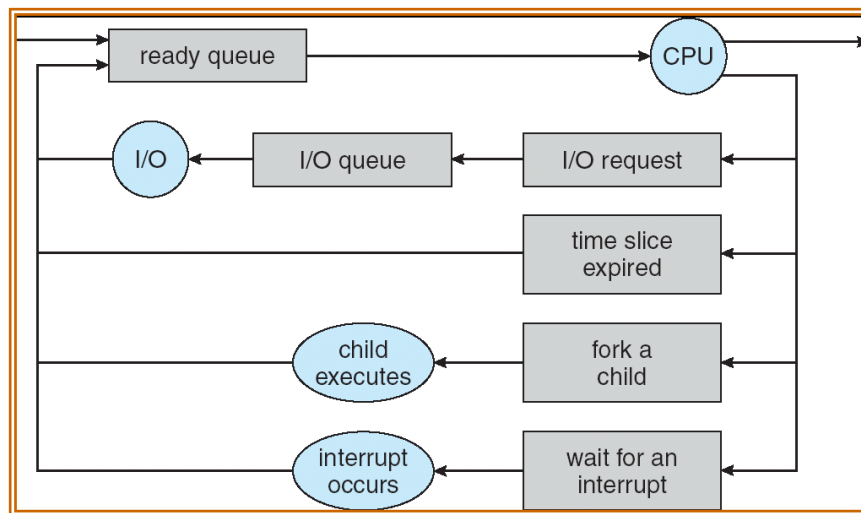


5. Process Scheduling

5.1 Scheduling Queues

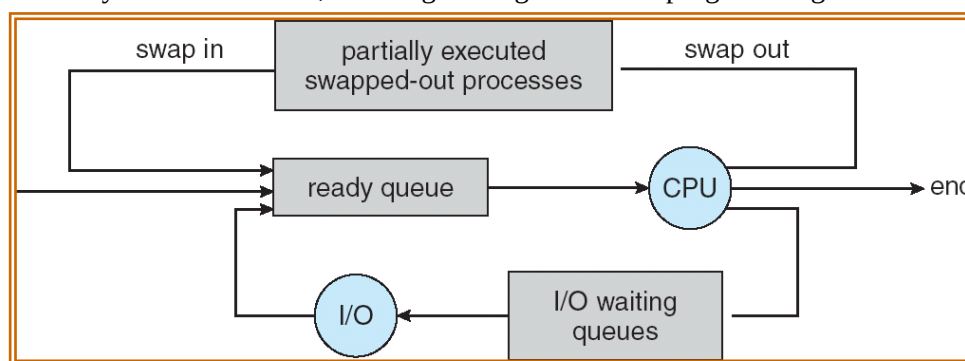
- Job queue — set of all processes in the system
- Ready queue — set of all processes residing in main memory, ready and waiting to execute (usually a linked list of PCBs)
- Device queues — set of processes waiting for a particular I/O device

Processes migrate among these queues as they execute — e.g., from ready queue → CPU → (I/O request) → device queue → ready queue, or → (fork a child) → wait for child, or → (interrupt occurs) → back to ready queue.



5.2 Schedulers

- Long-term scheduler (job scheduler) — selects which processes should be brought into the ready queue from the job pool. Invoked infrequently (seconds/minutes), so it can afford to be slow. It controls the degree of multiprogramming.
- Short-term scheduler (CPU scheduler) — selects which process should be executed next and allocates the CPU. Invoked very frequently (milliseconds), so it must be fast.
- Medium-term scheduler — added in some systems to swap partially-executed processes out of memory and back in later, reducing the degree of multiprogramming when needed.



5.3 Process Behaviour

- I/O-bound process — spends more time doing I/O than computation; has many short CPU bursts
- CPU-bound process — spends more time doing computation; has few, very long CPU bursts

6. Operations on Processes

6.1 Process Creation

A parent process creates children processes, which in turn create other processes, forming a tree of processes. Each process is identified by a unique process identifier (PID).

Resource sharing options:

- Parent and children share all resources
- Children share a subset of the parent's resources
- Parent and child share no resources

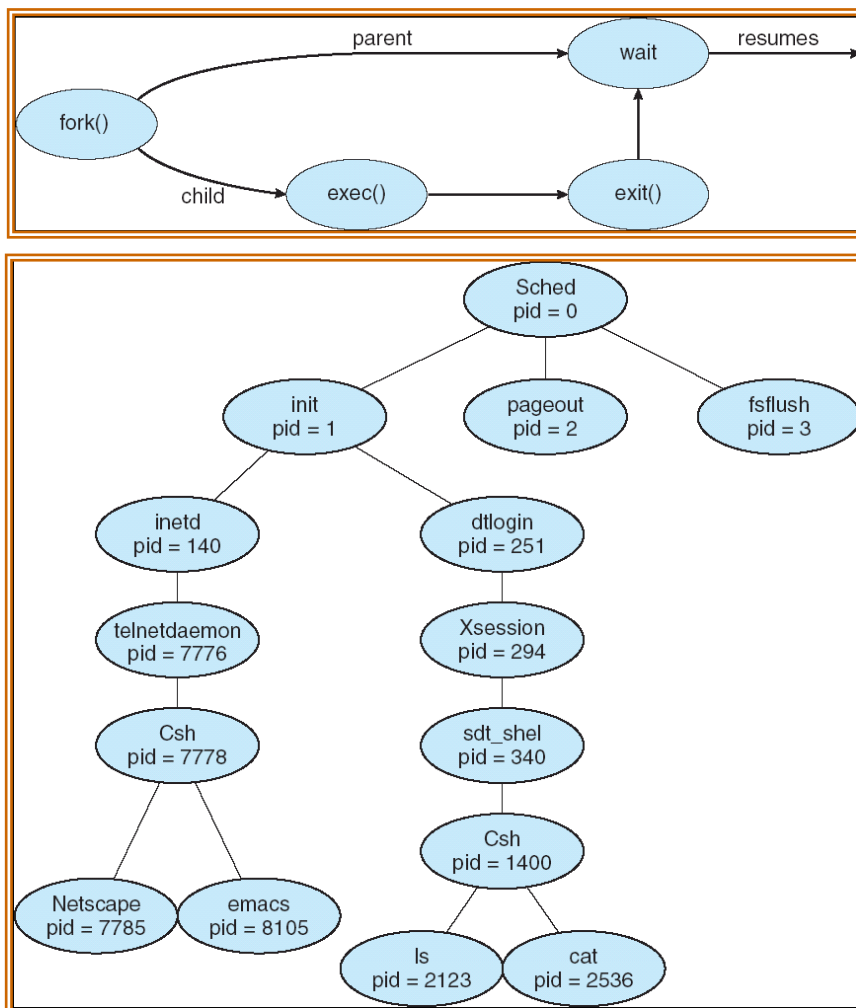
Execution options:

- Parent and children execute concurrently
- Parent waits until some or all children terminate

Address space options:

- Child is a duplicate of the parent (same program and data)
- Child has a new program loaded into it

UNIX example: the `fork()` system call creates a new process (a near-duplicate of the parent). The `exec()` system call is used after a `fork()` to replace the child's memory space with a new program. The parent typically calls `wait()` to pause until the child completes.



6.2 Process Termination

A process executes its last statement and asks the operating system to delete it, using the `exit()` system call:

- The process may return output data to the parent (via wait())
- The process's resources are deallocated by the operating system

A parent may also terminate the execution of a child process using abort(), typically because:

- The child has exceeded its allocated resources
- The task assigned to the child is no longer required
- The parent is exiting, and the OS does not allow a child to continue if its parent terminates — leading to cascading termination, where all descendants are terminated as well

7. Cooperating Processes

- Independent process — cannot affect, or be affected by, the execution of another process (no shared state)
- Cooperating process — can affect, or be affected by, the execution of another process (shares data/state)

Advantages of process cooperation:

- Information sharing
- Computation speed-up (parallel execution on multiple cores/processors)
- Modularity (dividing system functions into separate processes)
- Convenience (a user may run many tasks concurrently, e.g., editing, printing, compiling)

7.1 Producer–Consumer Problem

A classic paradigm for cooperating processes: a producer process produces information that is consumed by a consumer process.

- Unbounded buffer — places no practical limit on the size of the buffer
- Bounded buffer — assumes a fixed buffer size; the producer must wait if the buffer is full, and the consumer must wait if the buffer is empty

In the shared-memory bounded-buffer solution, a circular array buffer[BUFFER_SIZE] is used with in and out indices marking the next free slot and the next item to consume, respectively. This solution can only make use of BUFFER_SIZE – 1 elements (to distinguish a full buffer from an empty one).

- **Shared data**

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Bounded-Buffer – Insert() Method

```
while (true) {
    /* Produce an item */
    while (((in = (in + 1) % BUFFER_SIZE count) == out)
        ; /* do nothing -- no free buffers */
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
}
```

Bounded Buffer – Remove() Method

```
while (true) {  
    while (in == out)  
        ; // do nothing -- nothing to consume  
    // remove an item from the buffer  
    item = buffer[out];  
    out = (out + 1) % BUFFER SIZE;  
    return item;  
}
```

8. Interprocess Communication (IPC)

IPC is a mechanism for processes to communicate with each other and to synchronize their actions, typically via a message-passing system that avoids the need for shared variables.

An IPC facility provides two basic operations:

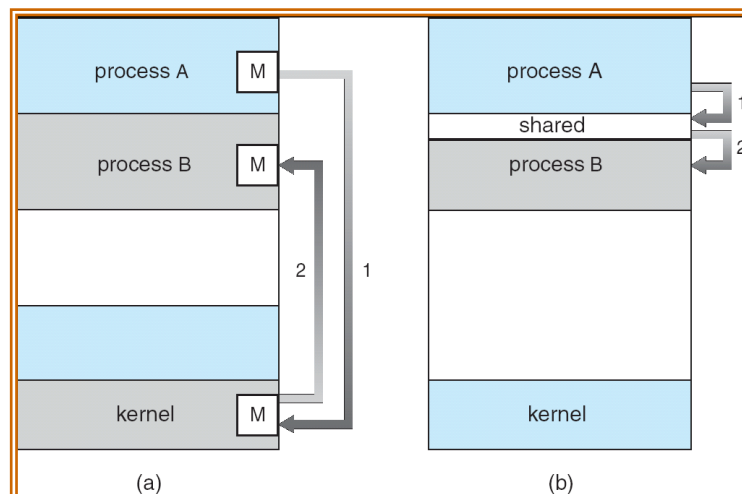
- send(message) — the message can be of fixed or variable size
- receive(message)

If processes P and Q wish to communicate, they need to:

- Establish a communication link between them
- Exchange messages via send()/receive()

A communication link can be implemented:

- Physically — e.g., shared memory, a hardware bus, or a network
- Logically — e.g., through logical properties such as direct/indirect communication, synchronous/asynchronous communication, and automatic/explicit buffering



9. Communication Models & Communication in Client–Server Systems

Two broad communication models exist:

- Message passing — processes exchange messages through the kernel, which relays the message (M) from one process's address space to another's, generally via two system calls (send/receive), each involving a trap to the kernel

- Shared memory — a region of memory is shared between processes, allowing them to exchange information by reading and writing data directly to that shared region, without the kernel's continued involvement once the shared segment is set up

These models generalize to communication in client–server systems, which can use sockets, remote procedure calls (RPC), or remote method invocation (RMI, in Java) depending on the level of abstraction required.

Multiple Choice Questions (10)

1. A process is best defined as:

- a) A program stored on disk
- b) A program in execution
- c) A collection of PCBs
- d) A scheduling queue

Answer: (b) A program in execution

2. Which of the following is NOT a component stored in a Process Control Block (PCB)?

- a) Program counter
- b) CPU registers
- c) Compiler source code
- d) I/O status information

Answer: (c) Compiler source code

3. When a process is waiting to be assigned to a processor, it is in the _____ state.

- a) New
- b) Running
- c) Waiting
- d) Ready

Answer: (d) Ready

4. The scheduler that selects which process should be executed next and allocates the CPU is the:

- a) Long-term scheduler
- b) Short-term scheduler
- c) Medium-term scheduler
- d) I/O scheduler

Answer: (b) Short-term scheduler

5. The long-term scheduler controls:

- a) The context-switch time
- b) The degree of multiprogramming
- c) The size of the PCB
- d) The buffer size in IPC

Answer: (b) The degree of multiprogramming

6. In UNIX, which system call is used after fork() to replace a child process's memory space with a new program?

- a) wait()
- b) exit()
- c) exec()
- d) abort()

Answer: (c) exec()

7. Cascading termination refers to:

- a) A process terminating itself using `exit()`
- b) All children of a process being terminated when the parent terminates
- c) A process being moved from ready to running state
- d) Two processes deadlocking each other

Answer: (b) All children of a process being terminated when the parent terminates

8. In the shared-memory bounded-buffer solution with `BUFFER_SIZE` slots, the maximum number of items that can be stored at once is:

- a) `BUFFER_SIZE`
- b) `BUFFER_SIZE + 1`
- c) `BUFFER_SIZE - 1`
- d) Unlimited

Answer: (c) `BUFFER_SIZE - 1`

9. Which of the following is an advantage of cooperating processes?

- a) Increased context-switch overhead
- b) Computation speed-up
- c) Reduced modularity
- d) Elimination of the need for IPC

Answer: (b) Computation speed-up

10. An IPC facility that allows processes to communicate primarily through `send()` and `receive()` operations, avoiding shared variables, is called a:

- a) Shared-memory system
- b) Message-passing system
- c) Scheduling queue
- d) Process control block

Answer: (b) Message-passing system

Five-Mark Questions (5)

1. Define a process and explain the different states a process passes through during its lifetime, with the help of a state diagram.
2. What is a Process Control Block (PCB)? List and explain the information it stores, and describe its role during a context switch.
3. Differentiate between the long-term scheduler, short-term scheduler, and medium-term scheduler. Why must the short-term scheduler be fast while the long-term scheduler can be slow?
4. Explain the process creation and termination mechanisms in UNIX using `fork()`, `exec()`, `wait()`, and `exit()`. Illustrate with a suitable example or diagram.
5. What is the Producer–Consumer problem? Describe the bounded-buffer shared-memory solution, including its `insert()` and `remove()` logic.
6. What is context switching? Why is it considered to be an overhead?